Easy — comprehension and demo follow-up

1. In one sentence, who is your user and what problem does your app solve for them?
Checks that the team can state a clear user story beyond feature lists — e.g. a traveler managing multi-currency spend (G2), an analyst triaging AML alerts (G3), or an investor linking news to holdings (G4).

2. What was your minimum viable product on day one, and what did you add in later increments?
Directly targets **Incremental Development & MVP Scoping**. Strong answers name a thin first slice (e.g. book one BUY trade, show one portfolio chart) and explain why later features were deferred.

3. Which external API did you integrate, and what does your system do with that data?
Every project has at least one integration (Twelve Data, Finnhub, Frankfurter, OpenSanctions, Alpaca, etc.). This confirms they understand the integration requirement, not just that they called an API.

4. How did you split roles across backend, frontend, and integration work?
Probes **Team Collaboration & Task Ownership** without putting anyone on the spot — e.g. G6's PRD-based split vs. G3's concentrated commit history.

5. Can you walk us through your main database entities and how they connect?
A natural follow-up to the data-model slide. Good answers stay at the whiteboard level: accounts, trades, holdings, alerts, news articles, etc., and why the schema was kept simple at first.

Medium — decisions, resilience, and process

6. What did you deliberately leave out of scope, and why?
Separates teams that scoped realistically (single user, demo profile, no cash ledger) from those that over-built or under-delivered. Expect honest tradeoffs: auth complexity, multi-currency, full OMS state machines.

7. What happens when your external data source is slow, rate-limited, or unavailable?
Core technical question for financial apps. Strong teams describe caching, fallback providers, stale-data labelling, or offline demo modes — e.g. Redis quote cache (G5), Alpaca → Twelve Data failover (G6), Frankfurter cache (G2/G3), Finnhub dual-key split (G4).

8. What was the hardest bug or integration issue you hit, and how did you resolve it?
Reveals real learning vs. rehearsed slides. Good answers mention JDK/Lombok issues, API quota exhaustion, CORS, Flyway migrations, or demo timing races.

9. How did you use branches, pull requests, and tags in git — and what would you improve?
Targets **Source Control Practices**. Useful contrast: G6's reviewed PR workflow vs. G3's single-author history vs. G1's backend work not fully on `main`.

10. If we skipped your UI and hit your REST API directly right now, which endpoint best proves the system works?
Tests whether the API is a first-class deliverable (Swagger, external-facing endpoints) or only a frontend backend. Bonus objective from the project brief.

Hard — architecture, risk, and judgment

11. Your project brief suggested a specific demo moment (API outage fallback, sanctions hold, news/price side-by-side, card freeze, etc.). How reliable is that live, and what is your backup plan if it fails?
Probes **Final Presentation & Demo Quality** under pressure. Strong teams have idempotent demo scripts, reset endpoints, or pre-cached data — not "we'll try again."

12. Where does your system stop short of making a financial or compliance decision automatically?
Separates thoughtful design from demo magic. Examples: AML priority score vs. determination (G3), sentiment vs. price alignment labelled DIVERGENT (G4), fraud alert vs. confirmed fraud (G2), MOCK vs. LIVE quote labelling (G5).

13. What data-integrity or concurrency risks exist in your design — duplicate payments, negative positions, double alerts, stale portfolio values — and how did you guard against them?
Hard technical judgment. Look for idempotency keys, transactional boundaries, no-short validation, append-only audit trails, or explicit "we didn't solve this yet."

14. If this went to a small pilot with real users in four weeks, what are the top three gaps you would close first?
Forces prioritization beyond feature wishlists: security hardening, CI/CD, observability, secrets management, E2E tests, rate-limit monitoring, licensing (e.g. OpenSanctions NC use).

15. Looking at another team's project today, what architectural choice did you make differently — and with another week, would you keep that choice or change it?
Hardest question: requires listening to other presentations and articulating tradeoffs (monolith vs. worker, rules engine vs. LLM sentiment, Redis cache vs. DB polling, SPA vs. plain HTML). Strong answers compare concrete alternatives, not generic "we'd add more tests."